

天津大学
智能与计算学部
网络空间安全学院

《软件安全》实验报告
实验四：API 自搜索技术

姓名：安孟喆

学号：3024244059

一. 实验环境

本实验在 VMware Workstation 虚拟机中的 Windows 10 操作系统上完成，开发工具为 Visual Studio，使用 C++ 控制台空项目。项目配置为 Debug、x86 (Win32) 平台。选择 x86 的原因是实验需要使用 32 位进程中的 FS 段寄存器，通过 FS:[0x30] 访问进程环境块 PEB；Visual Studio 的 x64 编译模式不支持本实验所用的内联汇编形式。

二. 实验目的

1. 理解 ASLR 地址空间布局随机化机制，认识在 Windows 10 中将 API 地址硬编码到程序或 Shellcode 中存在的稳定性。
2. 掌握通过 TEB、PEB 和 PEB_LDR_DATA 遍历当前进程已加载模块的方法。
3. 掌握 PE 文件导出表的解析过程，能够根据函数名称动态定位 LoadLibraryA 和 MessageBoxA 的入口地址。
4. 在不直接依赖固定 API 地址的情况下加载 user32.dll，并调用 MessageBoxA 弹出对话框。

三. 实验步骤

1. 创建工程并配置 x86 环境

在 Visual Studio 中创建名为 exp4 的 C++ 空项目，添加 main.cpp 文件，将解决方案平台由 x64 修改为 x86。首先调用系统函数输出 kernel32.dll、LoadLibraryW、程序函数和全局变量的地址，用于观察 Windows 10 进程中的地址分布并理解 ASLR 的作用。

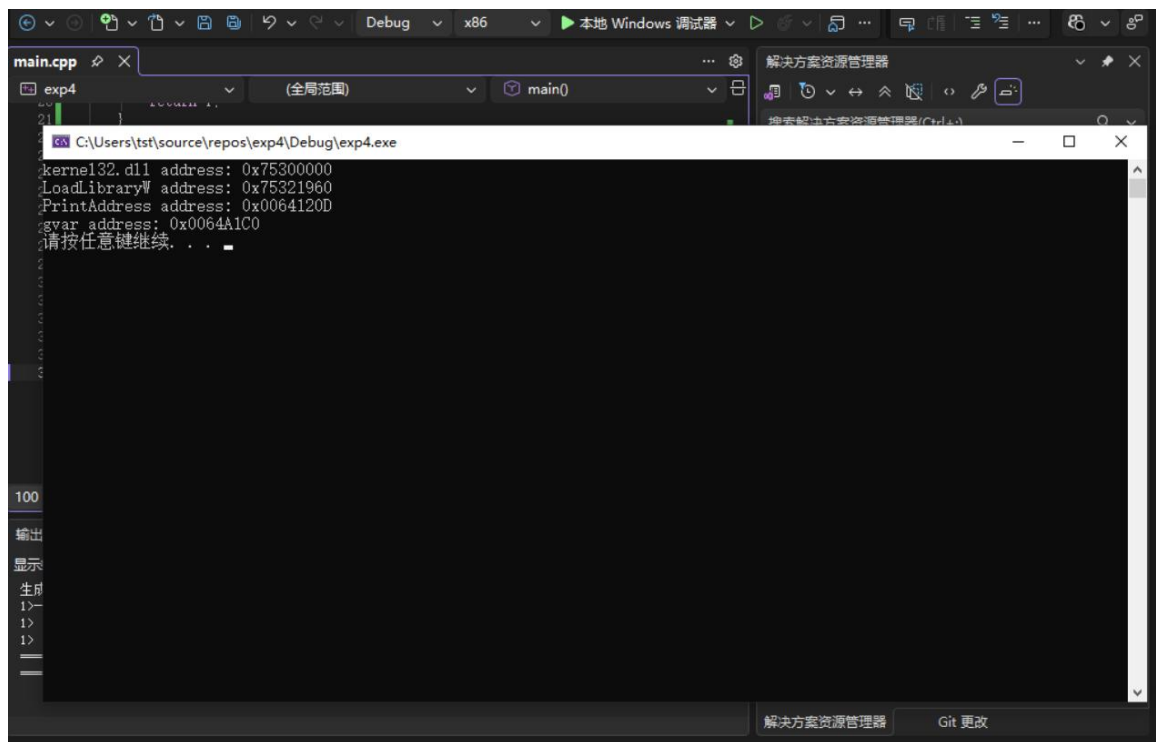


图 1 x86 环境下输出模块、API、程序函数和全局变量地址

2. 通过 FS:[0x30] 获取 PEB 地址

在 32 位 Windows 进程中, FS 段寄存器指向当前线程的 TEB。TEB 偏移 0x30 处保存当前进程 PEB 的指针, 因此可以使用内联汇编读取 PEB 地址。PEB 是进程级核心数据结构, 其中记录了加载器、进程参数和堆等信息。

```
__asm {  
    mov eax, fs:[0x30]  
    mov pebAddress, eax  
}
```

3. 遍历模块链表定位 kernel32.dll

从 PEB 基址偏移 0x0C 处取得 PEB_LDR_DATA 指针, 再从 PEB_LDR_DATA 偏移 0x0C 处获得 InLoadOrderModuleList 双向链表。程序依次遍历 LDR_DATA_TABLE_ENTRY 结点, 在 32 位结构中读取结点偏移 0x18 处的 DllBase 和偏移 0x2C 处的 BaseDllName, 并对模块名进行不区分大小写比较。当模块名为 kernel32.dll 时, 返回其实际加载基址。

程序同时调用 GetModuleHandleA("kernel32.dll") 进行结果校验。该系统函数只用于验证, 自搜索过程本身依靠 PEB 模块链表完成。实验运行中两种方法均得到 0x75300000, 说明模块定位正确。

4. 解析 PE 导出表并搜索 LoadLibraryA

得到 kernel32.dll 基址后，将其视为内存中的 PE 映像。首先检查 IMAGE_DOS_HEADER 的 MZ 标志，并利用 e_lfanew 找到 IMAGE_NT_HEADERS32；然后从 OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT] 取得导出表 RVA。导出表中的 AddressOfNames、AddressOfNameOrdinals 和 AddressOfFunctions 分别指向函数名称表、名称序号表和函数地址表。

程序遍历名称表，将名称与 LoadLibraryA 比较；匹配后通过名称序号表取得函数地址表下标，再读取函数 RVA。最终入口地址按“模块加载基址 + 函数 RVA”计算。Windows 10 中部分 kernel32.dll 导出项可能是转发导出，程序还会解析形如 KERNELBASE.LoadLibraryA 的转发字符串，并在目标模块中继续搜索。

```
WORD index = functionOrdinals[i];
DWORD functionRva = functionAddresses[index];
FARPROC functionAddress = (FARPROC)(base + functionRva);
```

5. 加载 user32.dll 并搜索 MessageBoxA

将自搜索获得的 LoadLibraryA 地址转换为符合 WINAPI 调用约定的函数指针，调用 LoadLibraryA("user32.dll") 加载 user32.dll。随后采用相同的 PE 导出表解析方法，在 user32.dll 的导出函数中搜索 MessageBoxA，并将得到的地址转换为消息框函数指针。

```
HMODULE user32Base = MyLoadLibraryA("user32.dll");
FARPROC messageBoxAddress = FindExportByName(user32Base, "MessageBoxA");
MyMessageBoxA(NULL, "API self-search succeeded!", "Experiment 4",
               MB_OK | MB_ICONINFORMATION);
```

6. 运行程序并验证实验结果

运行程序后，控制台依次输出 PEB 地址、kernel32.dll 基址、LoadLibraryA 地址、user32.dll 基址和 MessageBoxA 地址。自搜索结果分别与 GetModuleHandleA、GetProcAddress 的返回值完全一致，随后程序成功弹出标题为“Experiment 4”的消息框。具体结果如下：

1. PEB 地址为 0x00C47000。
2. kernel32.dll 自搜索基址与 GetModuleHandleA 返回值均为 0x75300000。

3. LoadLibraryA 自搜索地址与 GetProcAddress 返回值均为 0x75320E70。
4. user32.dll 的加载基址为 0x76FD0000。
5. MessageBoxA 自搜索地址与 GetProcAddress 返回值均为 0x77051230。

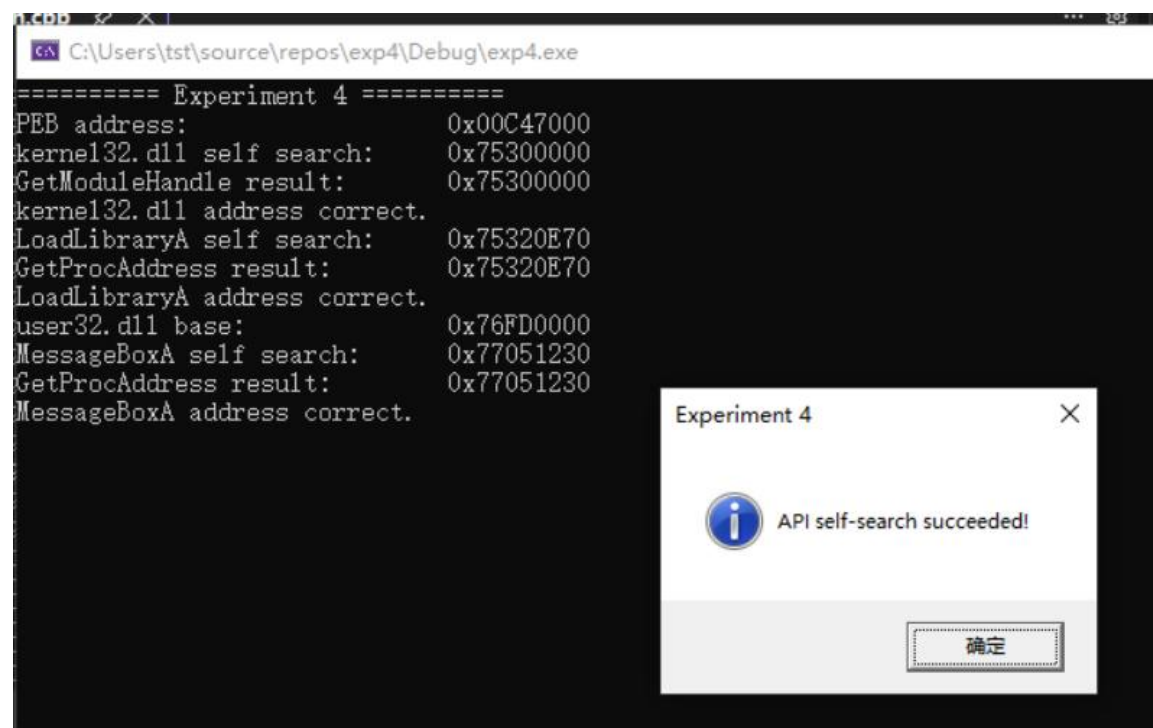


图 2 API 自搜索地址验证及 MessageBoxA 对话框运行结果

7. 实验结果分析

实验结果表明，程序没有把 LoadLibraryA 或 MessageBoxA 的绝对地址写死，而是在进程运行时从 PEB 获取模块信息，再解析内存中 PE 映像的导出表得到 API 入口地址。因此，即使 Windows 10 的 ASLR 使 DLL 基址在不同系统启动或不同环境中发生变化，程序仍可依据当前进程中的真实加载地址完成函数定位。

GetModuleHandleA 和 GetProcAddress 的输出仅作为对照验证。自搜索地址与系统 API 返回地址一致，且 MessageBoxA 能够被正常调用，说明 PEB 模块遍历、PE 导出表解析、转发导出处理和函数指针调用均正确完成。

四. 心得体会

通过本次实验，我对 Windows 进程内部的 TEB、PEB、PEB_LDR_DATA 和模块链表之间的关系有了更直观的理解。以前调用 API 时只需要写出函数名，

本实验则进一步分析了操作系统如何记录已加载 DLL，以及程序如何从 PE 导出表将函数名称解析为实际入口地址。

实验中最关键的两点是平台位数和地址换算。只有将工程设置为 x86，FS:[0x30] 及相应的 32 位结构偏移才有效；导出表中保存的是 RVA，必须与 DLL 的当前加载基址相加才能得到可调用的虚拟地址。此外，Windows 10 中还存在转发导出，说明简单读取 kernel32.dll 的函数 RVA 并不一定能够直接得到最终实现地址，需要继续解析转发目标。

本实验也说明 ASLR 会提高固定地址利用方式的失败概率，但运行中的程序仍可根据系统数据结构动态获取模块和函数地址。完成弹窗验证后，我掌握了 API 自搜索的完整流程，也加深了对 Windows 加载机制、PE 文件结构及安全防护机制之间关系的认识。